Midterm – Part 2

How The Times were Measured

Each program (sobel_pthread.c and sobel_openmp.c) runs the original sequential Sobel first, then the parallel version on the same image, and checks that both outputs match byte by byte. Every run printed "Parallel output matches sequential: YES". Only the Sobel convolution is timed. Loading the image and saving the .bmp are not included. The pthread version is timed with clock_gettime(CLOCK_MONOTONIC). It measures wall clock time and is not affected by changes to the system clock. The timer starts before the threads are created and stops after all of them are joined, so thread creation is included in the time. CPU time functions like clock() would be wrong here because they add up the time of every thread, which makes the parallel version look slower than it really is. The OpenMP version is timed with omp_get_wtime(), which is OpenMP's own wall clock timer. It is called right before and after the parallel for loop, so starting the thread team is included. NGGYU.jpg is 360 x 360 pixels and NGGYU2.jpg is 1170 x 2532 pixels (about 23 times more pixels). Since every run also prints a sequential time, the Sequential column is the average of the 10 sequential times recorded for each image. All times are in seconds. All tests were run on a computer with 16 threads.

Results

			Number of Threads				
Sobel Edge Detection	Image	Sequential	1	2	3	4	8
pthread time	NGGYU.jpg	0.013445	0.013757	0.007874	0.004967	0.003509	0.00301
	NGGYU2.jpg	0.309914	0.33676	0.16013	0.107782	0.081178	0.048839
OpenMP Time	NGGYU.jpg	0.013445	0.013666	0.007418	0.004577	0.003803	0.003014
	NGGYU2.jpg	0.309914	0.295153	0.160606	0.111312	0.087914	0.048607

Speedup = sequential average time / parallel time

			Number of Threads				
Sobel Edge Detection	Image	Sequential	1	2	3	4	8
pthread time	NGGYU.jpg	1.00x	.98x	1.71x	2.71x	3.83x	4.47x
	NGGYU2.jpg	1.00x	.92x	1.94x	2.88x	3.82x	6.35x
OpenMP Time	NGGYU.jpg	1.00x	.98x	1.81x	2.94x	3.54x	4.46x
	NGGYU2.jpg	1.00x	1.05x	1.93x	2.78x	3.53x	6.38x

Observations

1 thread is about the same as sequential, or a little slower. With 1 thread, both versions do the exact same work as the sequential code, plus the cost of creating a thread (pthreads) or starting a parallel region (OpenMP). That is why the 1 thread times are within a few percent of the sequential time. The OpenMP 1 thread run on NGGYU2.jpg came out slightly faster (1.05x), which is just run to run variation since the sequential times for that image ranged from 0.292 s to 0.327 s.

The large image scales almost perfectly from 1 to 4 threads. On NGGYU2.jpg, 2 threads gave about 1.9x, 3 threads about 2.8x, and 4 threads about 3.5x - 3.8x. Sobel is an embarrassingly parallel problem. Every output pixel only reads from the input image and only writes to its own spot in the output, so the threads never have to wait on each other. There are no locks, no barriers and no shared writes, so the only extra cost is starting and joining the threads.

8 threads was still faster, but less efficient. Going from 4 to 8 threads took NGGYU2.jpg from about 0.08 s to 0.049 s (6.4x). My computer has 16 hardware threads, so all 8 threads could run at the same time without taking turns on the same core. The efficiency (speedup / threads) still dropped from about 95% at 4 threads to about 80% at 8 threads. Some reasons for this are that the 16 hardware threads likely come from 8 physical cores with hyperthreading, so some of the 8 threads may end up sharing a physical core; all of the threads are reading the input and writing the output at the same time, so memory bandwidth becomes more of a limit; and the cost of creating threads is a bigger part of a shorter run.

The small image does not benefit as much. On NGGYU.jpg, 4 threads still gave close to 4x, but 8 threads only reached about 4.5x (56% efficiency). The whole sequential run only takes about 0.013 s, so at 8 threads each thread only has about 45 rows to process and finishes in a couple of milliseconds. At that point, the time to create and join the threads is no longer small compared to the actual work. The small image times are also less reliable in general, because a tiny amount of noise from the operating system is a large percent of a 3 ms run.

pthreads and OpenMP performed about the same. At 8 threads both versions took 0.049 s on NGGYU2.jpg and 0.003 s on NGGYU.jpg. This makes sense because both split the work in a very similar way. The pthread version gives each thread a continuous block of rows, and OpenMP with collapse(2) and schedule(static) gives each thread a continuous block of pixels, which ends up being almost the same split. The pthread version was a little faster at 4 threads on both images, but the difference is small enough that it is most likely run to run variation. The main difference between them is how much code it took. OpenMP only needed one #pragma line, while pthreads needed a struct, a thread function, and code to split up the rows and handle the remainder.

Why collapse(2) and the row remainder matter. For OpenMP, collapse(2) turns the row and column loops into one loop of about 2.95 million pixels for NGGYU2.jpg, so OpenMP can divide the work very evenly no matter how many threads are used. For pthreads, NGGYU2.jpg has 2530 interior rows, which does not divide evenly by 3, 4 or 8. Instead of giving all of the leftover rows to the last thread, the first few threads each take one extra row, so no thread does more than one extra row of work. Because the work is balanced this way, no thread is left waiting on a slower one at the join.

Conclusion

Both parallel versions produce the same output as the sequential program and give a large speedup on the bigger image, up to about 6.4x with 8 threads. Sobel edge detection works very well in parallel because each pixel is independent. The speedup is best when the image is large enough that the actual work is much bigger than the cost of creating threads, and it starts to level off as more threads share the same memory and physical cores.